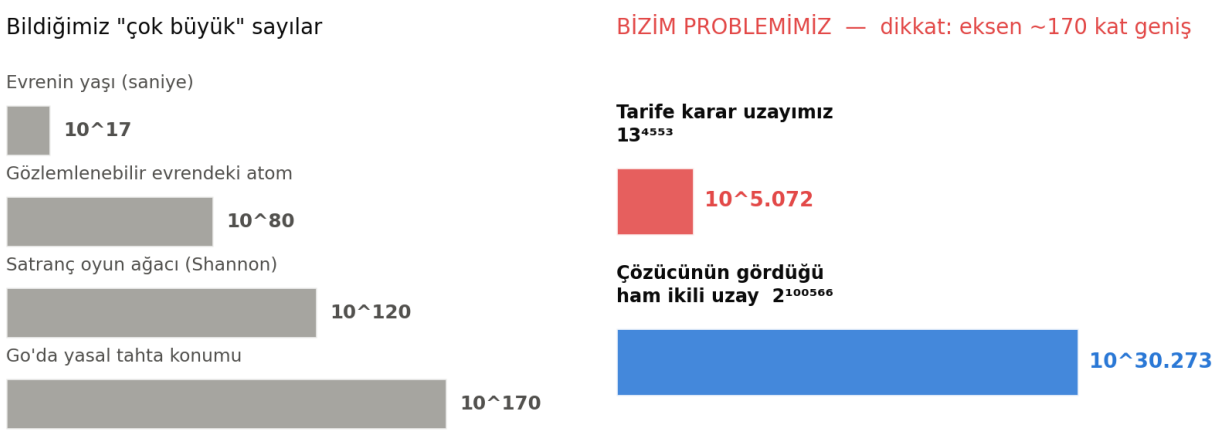
TEKNOFEST · Yapay Zekâ Destekli Havayolu Optimizasyonu

PROBLEMİN BOYUTU ve ÇÖZÜM ALTYAPISI

Kaç ihtimal var · neyi değiştiriyoruz · Pyomo ve çözücü ne işe yarıyor?

Takım: Büke



çubuk uzunluğu = 10'un ÜSSÜ (sayının kendisi değil)

	Değer
Ham girdi verisi	**89.094** satır · 4 Excel dosyası
Türetilmiş veri katmanı	**118.177** satır · 9 CSV
Değiştirdiğimiz karar	**4.553** uçuş olayının IST saati (±60 dk / 10 dk ızgara)
Olay başına seçenek	**13**
Tarife karar uzayı	$13^{4553} \approx 10^{5.072}$ ihtimal
Model boyutu	**100.566** ikili değişken · **70.359** kısıt
Modelleme çerçevesi	**Pyomo** (problemi tarif eder)
Çözücü	**HiGHS** (problemi çözer ve optimalite kanıtı üretir)
Sonuç	**%+4,72** rekabetçilik · global MIP-gap **%1,18**

Tüm sayılar projenin gerçek çıktı ve veri dosyalarından hesaplanmıştır.

İçindekiler

#	Bölüm	İçerik
1	Bizden tam olarak ne isteniyor?	değiştirdiğimiz tek şey · dokunmadıklarımız · amaç fonksiyonu
2	Elimizde kaç adet veri var?	4 girdi dosyası · derived/ katmanı · output/ · veri hunisi
3	Kaç ihtimal var?	13^{4553} hesabı · karşılaştırmalar · uzayı neyle küçültüyoruz
4	Pyomo nedir, ne işe yarar?	modelleme ↔ çözme ayrımı · üç işi · ne DEĞİLDİR
5	Çözücü nedir, nasıl çalışır?	LP gevşemesi · dallanma-sınırlama · kesme · sezgisel · MIP-gap
6	Hangi çözücü, neden?	Gurobi · CPLEX · HiGHS · CBC karşılaştırması
7	`appsi` katmanı ve warm-start	kalıcı arayüz · setSolution · 08 neden buna bağımlı
8	Modelin gerçek boyutu	100.566 ikili değişken dökümü · kısıt sayıları · sonuç
A	Ek A — tek sayfalık sayı künyesi	tüm rakamlar tek tabloda
B	Ek B — iki konunun adım adım anlatımı	823 sabit olay neden kımıldamıyor · huni figürü satır satır

1 Bizden tam olarak ne isteniyor?

Bu sorunun cevabı şaşırtıcı derecede dardır ve projeyi anlamanın anahtarı budur: **değiştirdiğimiz tek bir şey var.**



Şekil 1.1 — Karar alanı ile veri alanının ayrımı. Sağ sütundaki hiçbir şeye dokunulmaz; hepsi girdi kabul edilir.

1.1 Değiştirdiğimiz: THY uçuşlarının IST saatleri

Bir uçuşun THY tarifesindeki iki "IST olayı" vardır:

- **Varış olayı** — o uçuş IST'e kaçta iniyor? (`Arr Time`, kodda rol = `arr`)
- **Kalkış olayı** — o uçuş IST'ten kaçta kalkıyor? (`Dep Time`, rol = `dep`)

Modelin oynattığı **tek** şey bu saatlerdir. İki sınırla:

Sınır	Değer	Anlamı
Pencere (Wdev)	±60 dk	Bir uçuş kendi mevcut saatinden en fazla 1 saat öne/arkaya çekilebilir
Izgara (Δ)	10 dk	Kaydırma yalnız 10'un katı olabilir → olay başına 13 seçenek

1.2 Değiştirmediğimiz: her şey

Aşağıdakiler **girdi**dir, karar değil. Hiçbiri modelin çıktısında farklı bir değer alamaz:

Dokunulmayan	Neden karar değil?
Uçuş rotaları, destinasyonlar	Ağ tasarımı bu problemin kapsamı dışında; brief tarife saatlerini soruyor
Uçuş sayısı, frekanslar	Yeni uçuş eklemek/çıkarmak filo ve slot kararıdır
Uçak tipi, koltuk kapasitesi	Amaç fonksiyonunda yer almıyor
Bilet fiyatları	Rekabetçilik burada **süre** üzerinden ölçülüyor, fiyat üzerinden değil
Uçuş süreleri (`legsum`)	Fiziksel gerçek — `ElapsedTime1 + ElapsedTime2`, veriden gelir ve sabittir

Dokunulmayan	Neden karar değil?
Uçak rotasyon eşleşmeleri	`Flight Pairs.xlsx`'ten okunur; hangi uçağın hangi turu yaptığı verilmiştir
Rakiplerin tarifeleri	49.467 rakip itinerary'si sabit referanstır — onları kıyas için kullanırız
`data/` altındaki dört Excel	Salt-okunur; proje politikası gereği asla yazılmaz

Bu darlık bir kısıtlama değil, bir güç

Karar alanı dar olduğu için **her iyileşme doğrudan tarife saatlerine atfedilebilir**. "Sonuç %4,72 arttı" dediğinizde bunun kaynağı belirsiz değildir: 1.067 uçuş hareketinin ortalama 19 dakika kaydırılması. Uçuş eklemek ya da fiyat oynatmak gibi başka kaldıraçlar karışmadığı için kazanç temiz ve savunulabilirdir.

1.3 Peki neyi maksimize ediyoruz?

Amaç fonksiyonu iki ödülün gelir-ağırlıklı toplamıdır:

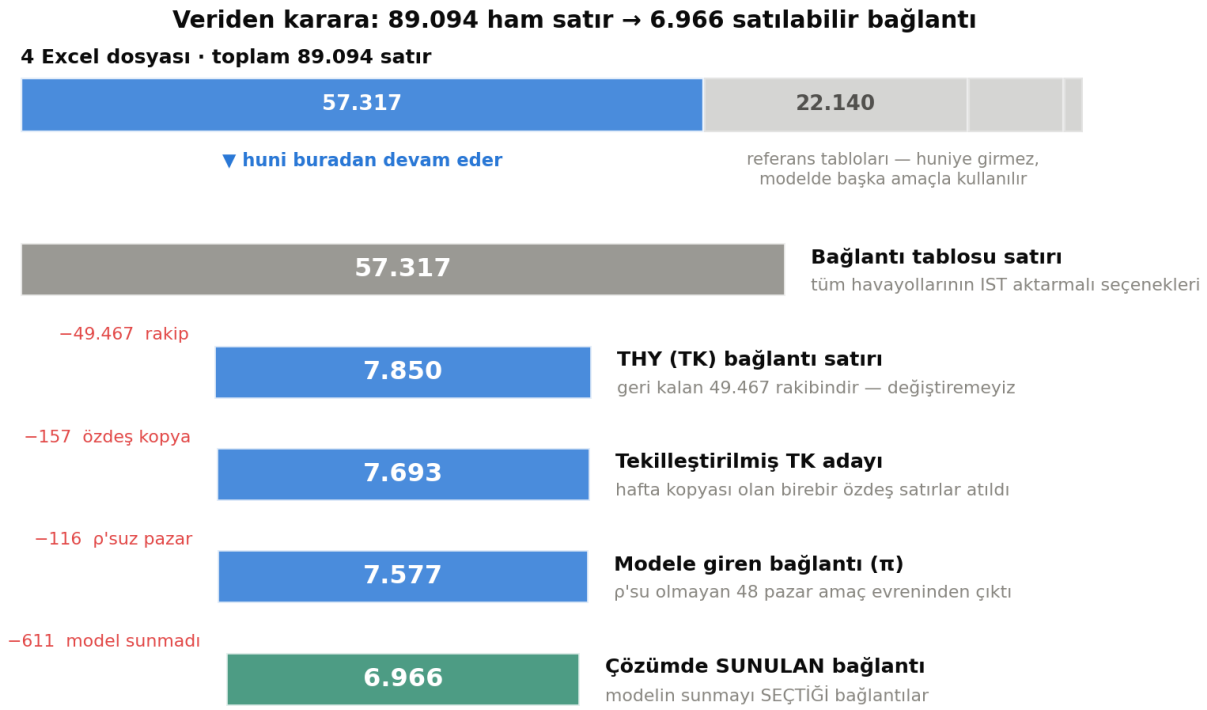
Amaç = Σ_pazar ρ · (aktarma ödülü + sıralama ödülü) − λ · Σ|kaydırma|

ρ (rho) — pazarın gelir/önem ağırlığı (3 ile 1.000 arası). Büyük pazarda kazanılan puan daha değerlidir.
Aktarma ödülü — o pazarda sunulan her geçerli bağlantı için azalan getirili puan: 1, ½, ¼, …
Sıralama ödülü — kaç rakibi geçtiğinize göre `change_ranking_input.xlsx` tablosundan okunan puan.
λ · kaydırma — gereksiz tarife değişimini engelleyen ceza (λ = 0,3).

Raporlanan tüm sonuçlar **cezasız (saf) rekabetçilik** üzerindendir; λ yalnızca hangi kaydırmaların yapılacağını süzer, puanı ölçmez.

2 Elimizde kaç adet veri var?

Sayılara girmeden önce genel resim: dört Excel dosyasındaki 89.094 satır, adım adım süzülerek 6.966 satılabilir bağlantıya dönüşüyor.



Şekil 2.1 — 89.094 ham satırdan 6.966 satılabilir bağlantıya. Her daralma bir modelleme kararıdır.

Figürü okurken dikkat

Üstteki şerit **eleme değil, dağılım** gösterir: 89.094 ham satırın 57.317'si bağlantı tablosudur ve huniye o girer. Kalan 31.777 satır (ödül tablosu, ρ tablosu, rotasyon eşleri) atılmaz — modelde başka amaçlarla sonuna kadar kullanılır. Alttaki çubuklar ise gerçek süzülmedir; her çubuk bir üstündekinin alt kümesidir. **Her adımın ayrıntılı anlatımı için bkz. Ek B.**

2.1 Ham girdi — 4 Excel dosyası, 89.094 satır

Dosya	Satır	Sütun	İçerik
`O&D Rakip Bağlantı Tablosu_updated.xlsx`	**57.317**	17	Tüm havayollarının IST aktarmalı bağlantıları
`change_ranking_input.xlsx`	**22.140**	4	(rakip sayısı, ilk sıra, son sıra) → ödül ağırlığı
`Yolcu Verisi_masked.xlsx`	**8.075**	3	O–D pazarı → gelir/önem katsayısı ρ
`Flight Pairs.xlsx`	**1.562**	4	Uçuş no → uçak rotasyon eşi (Pair)
TOPLAM	**89.094**	—	—

2.2 Bağlantı tablosunun içi

Ölçüt	Değer
Toplam satır (itinerary)	57.317
Bunun **THY (TK)** olanı	**7.850**
Rakip olanı	49.467
Farklı taşıyıcı sayısı	88 (TK dâhil) — rakip 87
Operasyon günü	**7** (1–7); takvim aralığı 3–11 Ağustos 2025
Benzersiz yönlü O–D pazarı	**1.408**
Benzersiz (pazar, gün) anahtarı	8.864
TK bağlantısı **olan** pazar	805
Yalnız rakibin olduğu pazar	603
Yalnız TK'nın olduğu pazar (rakipsiz)	266
Benzersiz TK uçuş numarası	**952** (480 gelen + 498 giden)
Mevcut tarifede [60,300] penceresinde olan TK bağlantısı	6.986 / 7.850 (**%89,0**)
Penceresi çok uzun olan (> 300 dk)	864
Penceresi çok kısa olan (< 60 dk)	0

"%89'u zaten uygun" — o hâlde iyileştirme nereden geliyor?

Uygun olmak yeterli değil; **rakipten hızlı** olmak gerekiyor. Bir bağlantı pencerede olabilir ama toplam yolculuk süresi (J) rakibinkinden uzunsa sıralama puanı getirmez. Optimizasyon iki şeyi birden yapar: pencere dışındaki 864 bağlantının bir kısmını içeri çeker **ve** zaten uygun olanların boşluğunu kısaltarak J'yi düşürüp rakip geçer.

2.3 Diğer üç tablonun içi

Tablo	Ölçüt	Değer
Yolcu Verisi	Benzersiz O–D	8.066 (8 tekrarlı satır ayıklandı)
	ρ aralığı	**3 – 1.000**
	Bağlantı tablosunda olup ρ'su **olmayan** pazar	**48** → amaç evreninden dışlandı
Ranking lookup	Satır	22.140
	Desteklediği maks. rakip sayısı N	**40**
	Ağırlık aralığı	−0,9744 … +1,6321
	Sıralamanın kötüleştiği satır	10.660 (tablo cezayı da tanımlıyor)
Flight Pairs	Satır / benzersiz uçuş no	1.562 / 1.381
	Benzersiz Pair (rotasyon grubu)	708
	TK uçuş no'su olup **partneri olmayan**	**176** → bu uçuşlar sabit kabul edildi

Rakip sayısı N için **taşıyıcı bazlı** sayım seçildi. Gerekçe veriden geldi: itinerary bazlı sayımda 73 pazar-gün 40'ı aşıyor (ödül tablosunun dışına çıkıyor), taşıyıcı bazlı sayımda ise en yüksek değer **17** — tablo tam kapsıyor. Ortalama N = 3,45.

2.4 Türetilmiş veri katmanı — `derived/`, 118.177 satır

Dosya	Satır	İçerik
`competitor_itineraries.csv`	49.467	Rakiplerin tüm bağlantı seçenekleri
`competitor_times_carrier.csv`	23.975	(pazar, gün, taşıyıcı) → en iyi J
`ranking_lookup.csv`	22.140	Sıralama ödül tablosu
`tk_connections.csv`	**7.693**	TK bağlantı adayları (tekilleştirilmiş)
`market_n_od.csv`	6.953	(pazar, gün) → rakip taşıyıcı sayısı
`flight_block_times.csv`	5.471	(uçuş, gün, rol) → bacak blok süresi
`markets.csv`	1.408	Pazar + ρ + kapsam bayrakları
`flight_rotation.csv`	894	TK uçuşlarının rotasyon eşleri
`tk_flights_without_partner.csv`	176	Sabit kabul edilen uçuş numaraları
TOPLAM	**118.177**	—

7.850 TK satırı neden 7.693 oldu? Hafta normalizasyonundan sonra **birebir özdeş** kalan 157 kopya satır atıldı. İkisini iki ayrı aday saymak, aynı ürüne çifte aktarma ödülü yazdırırdı.

2.5 Çıktı — `output/`, 17.317 satır

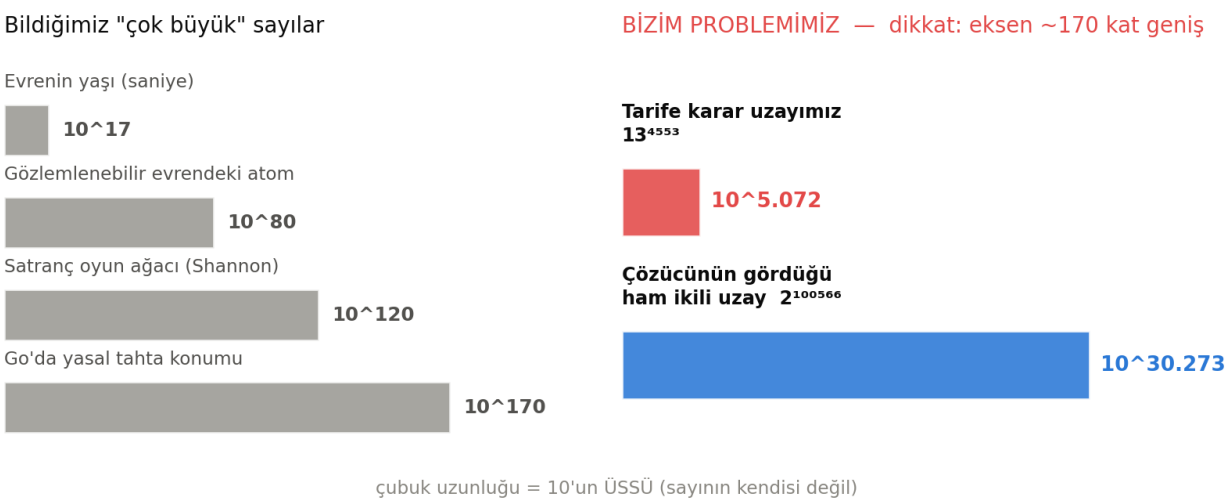
Dosya	Satır	İçerik
`selected_connections.csv`	**6.966**	Çözümde sunulan her bağlantı
`market_results.csv`	**4.660**	Her (pazar, gün) için sonuç ve ödül dökümü
`optimized_schedule.csv`	**4.554**	Her ayarlanabilir uçuş olayı: eski → yeni saat
`g_band_exemptions.csv`	1.132	Belgeli G düzenlilik muafiyetleri
`sweep_results.csv`	5	λ taraması sonuçları

3 Kaç ihtimal var?

3.1 Hesabın kendisi

Modelde **5.376 uçuş olayı** var. Bunların **823**'ü sabit (rotasyon partneri bilinmiyor → kımıldatılmıyor), **4.553**'ü ayarlanabilir. Her ayarlanabilir olayın 13 seçeneği var ve seçimler birbirinden bağımsız yapılabilir:

> **Tarife karar uzayı = 13 ^ 4.553 ≈ 10 ^ 5.072**
>
> Yani 1'in arkasına **beş bin yetmiş iki** sıfır. Bu sayı yazıya dökülemez; 10^{5072} ifadesinin kendisi bile ancak üstel gösterimle anlatılabilir.



Şekil 3.1 — Karşılaştırma. Sol paneldeki "devasa" sayılar, sağ panelin yanında çubuk olarak bile görünmez.

Sağ paneldeki iki çubuk iki **ayrı** sorunun cevabıdır — hangisinin "kaç ihtimal var?" sorusunu yanıtladığını bölüm 3.4'te netleştiriyoruz. Önce bu sayının nereden geldiğine ve ne kadar büyük olduğuna bakalım.

3.2 13'ün 4.553'üncü kuvveti niye? — çarpım kuralı

Sayı büyük olduğu için sihirli görünüyor ama hesabı ilkokul matematiği. Her ayarlanabilir olay için 13 seçenek var ve **her olay için ayrı ayrı** seçim yapılıyor. Küçükten başlayalım:

Kaç uçuş olayı?	Kaç farklı tarife?	Hesap
1	13	13
2	169	13 × 13
3	2.197	13 × 13 × 13
5	371.293	13^5
10	137.858.491.849 (≈138 milyar)	13^{10}
50	≈ 10^{56}	13^{50}
4.553	**≈ $10^{5.072}$**	**13^{4553}**

> **Neden toplama değil çarpma?**

Çünkü seçimler **birbirinden bağımsız** yapılıyor. İlk uçuş için 13 seçeneğin *her biri* için, ikinci uçuşun 13 seçeneği de ayrı ayrı denenebilir → 13 × 13. Üçüncü uçuş eklendiğinde bu 169'un her biri için yine 13 seçenek doğar → 2.197. Her yeni uçuş, olasılık sayısını **13 katına çıkarır**.

Peki uçuşlar birbirinden bağımsızsa neden her birini tek tek en iyilemiyoruz? İşte problemin asıl zorluğu burada: uçuşların **saat seçimi** bağımsız, ama **sonuçları** bağımsız değil.

- Bir gelen uçuşu kaydırınca ona bağlı onlarca bağlantının boşluğu değişir
- Aynı 10 dk kovasına giren uçuş sayısı değişir (**F** kapasitesi)
- Uçağın dönüş turu etkilenir (**A2** rotasyonu)
- O uçuşun diğer günlerdeki saatiyle uyumu bozulabilir (**G** düzenliliği)
- Pazarın ters yönüyle bağlantı dengesi bozulabilir (**E1**)

Tüm zorluk bu bağlaşıklıktan doğuyor

Uçuşlar gerçekten birbirinden bağımsız olsaydı her birini ayrı ayrı en iyilerdik: 4.553 × 13 = **59.189 deneme**, saniyeler sürerdi. Ama bir uçuşu oynatmak diğerlerinin puanını da değiştirdiği için hepsini **aynı anda** değerlendirmek zorundayız — kombinasyon sayısı işte o zaman patlıyor.

3.3 $10^{5.072}$ ne kadar büyük?

- Bu sayı yazıldığında **5.072 basamaklıdır** — normal punto ile yaklaşık iki sayfa dolusu rakam.
- Gözlemlenebilir evrendeki **atom sayısı** 10^{80}'dir. Bizim uzayımız bunun $10^{4.992}$ katıdır.
- Saniyede **bir milyar** kombinasyon deneyen bir bilgisayar, **Büyük Patlama'dan beri** çalışsaydı ancak $4{,}4 \times 10^{26}$ kombinasyon denerdi — yani işin $10^{-5.045}$'i kadarını. Pratikte **sıfır**.

Bu yüzden "hepsini dene" yaklaşımı bir seçenek değil. Ne bugün, ne de gelecekte herhangi bir bilgisayarla.

3.4 İkinci çubuk ne? Hangisi "gerçek cevap"?

Şekil 3.1'deki mavi çubuk kırmızıdan çok daha uzun. Bu, "asıl problem o" anlamına **gelmiyor**. İki çubuk iki ayrı sorunun cevabı:

	① $13^{4553} \approx 10^{5.072}$	② $2^{100.566} \approx 10^{30.273}$
Hangi soruyu yanıtlar?	**Kaç farklı TARİFE üretilebilir?**	Çözücünün formel arama kutusu ne kadar büyük?
Neyi sayar?	Yalnız uçuş saati seçimlerini (x)	**Tüm** ikili değişkenleri (x, y, z, w, β, γ)
Her nokta gerçek bir tarife mi?	**Evet** — her biri farklı bir tarife	Hayır — çoğu geçersiz veya x'ten zaten türetilmiş
"Kaç ihtimal var?" sorusunun cevabı	**BU**	Bu değil

Neden ikisi birden var?

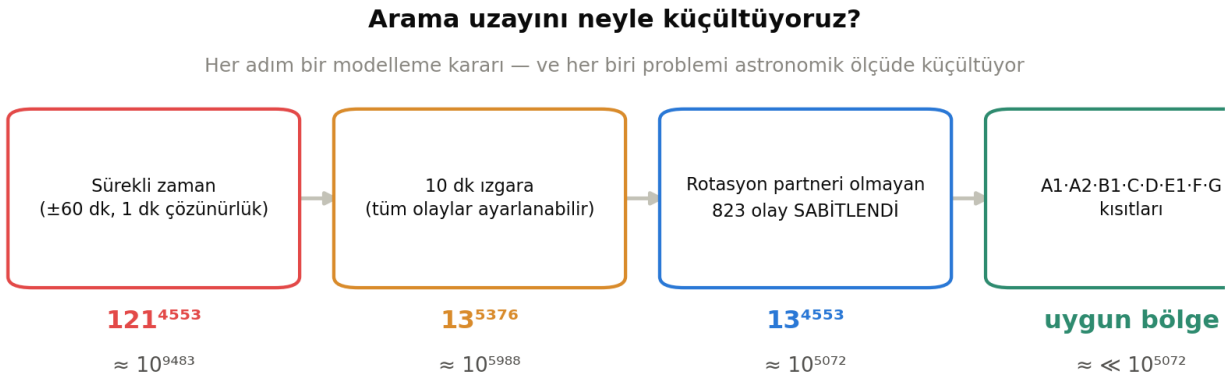
Model, "bu bağlantı sunuluyor mu?" (y), "bu rakip yenildi mi?" (β) gibi soruları da ikili değişkenle temsil eder — çünkü ödül mantığı ancak böyle **doğrusal** yazılabilir. Ama bu değişkenler serbest değildir: kısıtlar onları büyük ölçüde x'e kilitler. Saatleri seçtiğinizde hangi bağlantının uygun olduğu, hangi rakibin yenildiği zaten belli olur.

Yani ② sayısı çözücünün *çalışma alanının* boyutudur, gerçekten farklı çözümlerin sayısı değil. **"Kaç ihtimal var?" diye sorulduğunda verilecek cevap ①'dir: 13^{4553}.**

> **"Bunu bir for döngüsüyle çözemez miydik?"**
>
> Hayır. Saniyede bir milyar kombinasyon deneyen bir bilgisayar bile $10^{5.072}$ ihtimali bitiremez — Büyük Patlama'dan beri çalışsa işin $10^{-5.045}$'ini yapmış olurdu. Açgözlü (greedy) bir sezgisel yazabilirsiniz ve bir cevap alırsınız; ama o cevabın en iyiye ne kadar yakın olduğunu **asla** söyleyemezsiniz. MIP çözücünün verdiği %1,18'lik gap tam olarak bu eksiği kapatır.

Peki çözücü kaç noktayı gerçekten inceliyor? Milyarlar değil — dallanma ağacında **birkaç bin ile birkaç yüz bin arası düğüm**. $10^{5.072}$'nin yanında bu, hayal edilemeyecek kadar küçük bir oran. Nasıl başardığını Bölüm 5'te göreceğiz.

3.5 İhtimalleri neyle değiştiriyoruz?



Şekil 3.2 — Her modelleme kararı arama uzayını astronomik ölçüde küçültüyor.

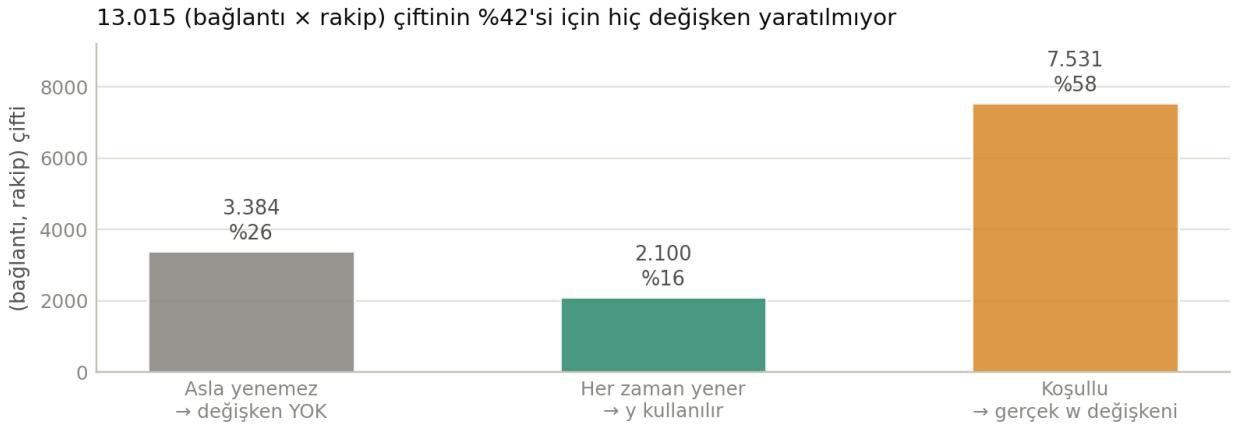
Dört kaldıraç var ve üçü doğrudan sizin elinizde:

Kaldıraç	Ne yapar?	Uzaya etkisi	Kodda
Izgara (Δ = 10 dk)	Sürekli zamanı 13 ayrık kovaya indirger	$121^{4553} \to 13^{4553}$ · yani 10^{3495} kat küçültür	`CFG["delta_min"]`
Pencere (Wdev = ±60)	Her uçuşu kendi saatine bağlar	Sınırsız olsa aday sayısı 144'e çıkardı	`CFG["wdev_min"]`
Sabit uçuşlar	823 olayın seçeneğini 1'e indirir	$13^{823} = 10^{917}$ kat küçültür	`tk_flights_without_partner.csv`
Kısıtlar (A1–G)	Uzayı küçültmez — içindeki hangi noktaların **geçerli** olduğunu belirler	Uygun bölge, uzayın çok küçük bir alt kümesi	`build_model()`

> **Kritik ayrım: uzayı küçültmek ≠ kısıt koymak**
>
> **Izgara ve pencere** arama uzayının *büyüklüğünü* değiştirir — daha az nokta vardır.
> **Kısıtlar** ise uzayın büyüklüğüne dokunmaz; hangi noktaların **uygun** olduğunu söyler. Aslında kısıtlar çözücünün işini *kolaylaştırır*: uygun bölge daraldıkça budanabilecek dal sayısı artar.

3.6 Model, işe başlamadan önce de eliyor

Çözücüye gitmeden, model kurulurken bile ciddi bir eleme yapılır. En belirgin örnek rakip yenme değişkenleridir. Teorik olarak her (bağlantı × rakip) çifti için bir değişken gerekir — toplam **13.015** çift. Ama kod üç duruma ayırır:



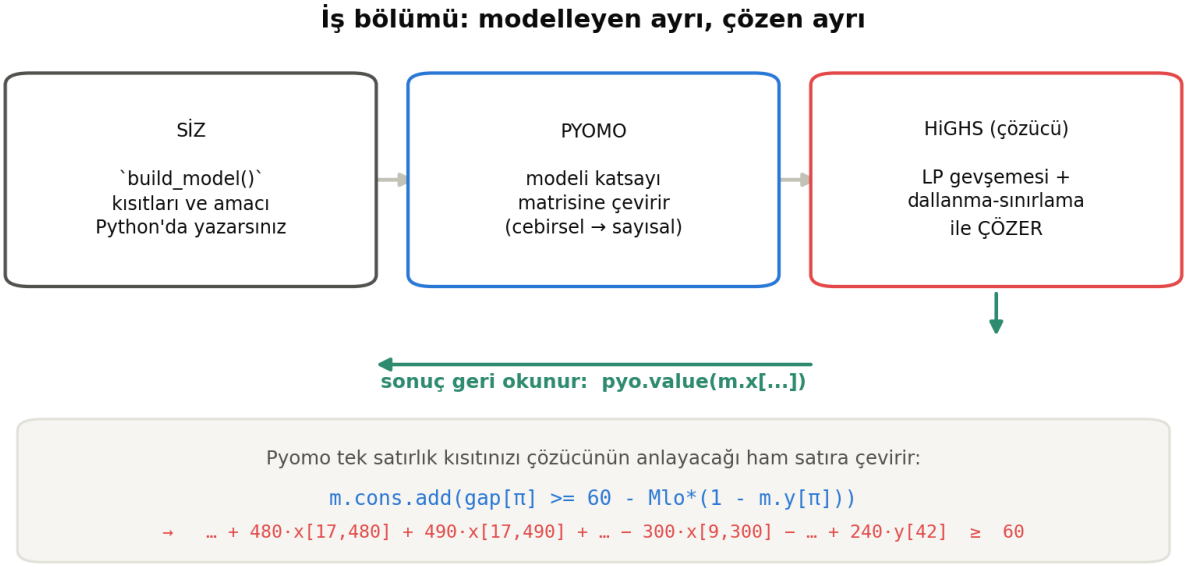
Şekil 3.3 — Çiftlerin %42'si için hiç değişken yaratılmıyor.

Durum	Koşul	Sayı	Ne yapılır?
Asla yenemez	İzin verilen boşluk, ulaşılabilir en küçüğün altında	**3.384** (%26)	Değişken **yaratılmaz**
Her zaman yener	Pencere içindeki her boşlukta rakip geçilir	**2.100** (%16)	`w` yerine doğrudan `y` kullanılır
Koşullu	Boşluğa bağlı — bazen yener, bazen yenmez	**7.531** (%58)	Gerçek `w` değişkeni + 2 kısıt

Benzer elemeler her yerde var: F kapasite kısıtı yalnız **bağlayıcı olabilecek** kovalara eklenir (1.395 kısıt), G düzenliliği penceresi yetişmeyen 4 grup için atlanır. Bunlar kozmetik değil — büyük MIP'lerde çözülebilirlik ile çözülemezlik arasındaki fark tam olarak bu tür ayıklamalardır.

4 Pyomo nedir, ne işe yarar?

Optimizasyonda iki tamamen farklı iş vardır ve bunları karıştırmamak gerekir:



Şekil 4.1 — İş bölümü. Siz problemi tarif edersiniz, Pyomo tercüme eder, çözücü çözer.

	Modelleme (Pyomo)	**Çözme (HiGHS)**
Soru	Problem **nedir**?	Problem **nasıl** çözülür?
Ürün	Değişkenler, kısıtlar, amaç	Optimal değerler + optimalite kanıtı
Dil	Python	C++ (siz görmezsiniz)
Sizin projede	`03_model.py` · `build_model()`	`highspy` paketi
Benzetme	SQL sorgusu	Veritabanı motoru

4.1 Pyomo'nun üç işi

1) Modeli Python'da ifade etmenizi sağlar

`src/03_model.py` · satır 360–375 · değişkenler ve A1 kısıtı

```
360     # --- değişkenler: x[eid,val] (yalnız ayarlanabilir olaylar) -------------
361     x_index = [(e["id"], v) for e in inst.ev_by_id.values() if e["adjustable"] for v in e["cands"]]
362     m.x = pyo.Var(x_index, domain=pyo.Binary)
363
364     # y_π : bağlantı sunuluyor (uygun) mu
365     P = list(range(len(inst.conns)))
366     m.y = pyo.Var(P, domain=pyo.Binary)
367     if CFG["B2_force_offer"]:
368         m.s = pyo.Var(P, domain=pyo.Binary)     # B2 ayrık "çok kısa / çok uzun" seçici
369
370     m.cons = pyo.ConstraintList()
371
372     # --- (A1) her ayarlanabilir olay tam bir kovaya atanır -----------------
373     for e in inst.ev_by_id.values():
374         if e["adjustable"]:
375             m.cons.add(sum(m.x[e["id"], v] for v in e["cands"]) == 1)
```

`pyo.Var`, `pyo.ConstraintList`, `pyo.Objective` — bunlar Pyomo'nun sunduğu yapılardır. Onlarla yazdığınız şey Python kodu gibi görünür ama aslında bir **matematiksel model tarifidir**.

2) Bunu çözücünün anlayacağı forma çevirir

Çözücü Python bilmez. Sadece sayılardan oluşan bir katsayı matrisi görür. Tek satırlık bir B1 kısıtınız:

`src/03_model.py` · satır 386–390 · B1 — Pyomo'da nasıl görünüyor

```
386          # --- (B1) y=1 ⟹ ℓ ≤ gap ≤ u  (yön-yerel sıkı M) ------------------
387          Mlo = max(0, L - gmin)
388          Mhi = max(0, gmax - U)
389          m.cons.add(gap[pi] >= L - Mlo * (1 - m.y[pi]))
390          m.cons.add(gap[pi] <= U + Mhi * (1 - m.y[pi]))
```

`gap[pi]` aslında `theta_expr(dep) − theta_expr(arr)` yani iki toplamın farkıdır. Pyomo bunu açar, düzleştirir ve çözücüye şuna benzer **tek bir satır** olarak verir:

> **… + 480·x[17,480] + 490·x[17,490] + … − 300·x[9,300] − 310·x[9,310] − … + 240·y[42] ≥ 60**
>
> Bu çeviriyi elle yapmak zorunda kalsaydınız `03_model.py` 1.357 satır değil, on binlerce satır olurdu — ve her kısıt değişikliğinde matris indekslerini elle güncellemek gerekirdi.

3) Sonucu geri okur

`src/03_model.py` · satır 1100–1101 · çözümün gerçek saatlere çevrilmesi

```
1100         new_time = {e["id"]: (round(sum(v * pyo.value(m.x[e["id"], v]) for v in e["cands"]))
1101                               if e["adjustable"] else e["cur"]) for e in inst.ev_by_id.values()}
```

Çözücü isimsiz bir sayı dizisi döndürür; Pyomo onu tekrar `m.x[eid, v]` gibi anlamlı isimlere bağlar. `pyo.value()` bu değeri okur.

4.2 Pyomo tam olarak ne DEĞİLDİR

- **Bir çözücü değildir.** Tek başına hiçbir şey çözemez; mutlaka arkasına bir çözücü takmanız gerekir.
- **Bir algoritma değildir.** Dallanma-sınırlama, kesme düzlemleri, sezgiseller — hiçbiri Pyomo'da yoktur.
- **Bir hızlandırıcı değildir.** Aynı modeli daha iyi *ifade etmenizi* sağlar; modelin kendisi kötüyse (gevşek big-M gibi) Pyomo bunu düzeltmez.

5 Çözücü nedir, nasıl çalışır?

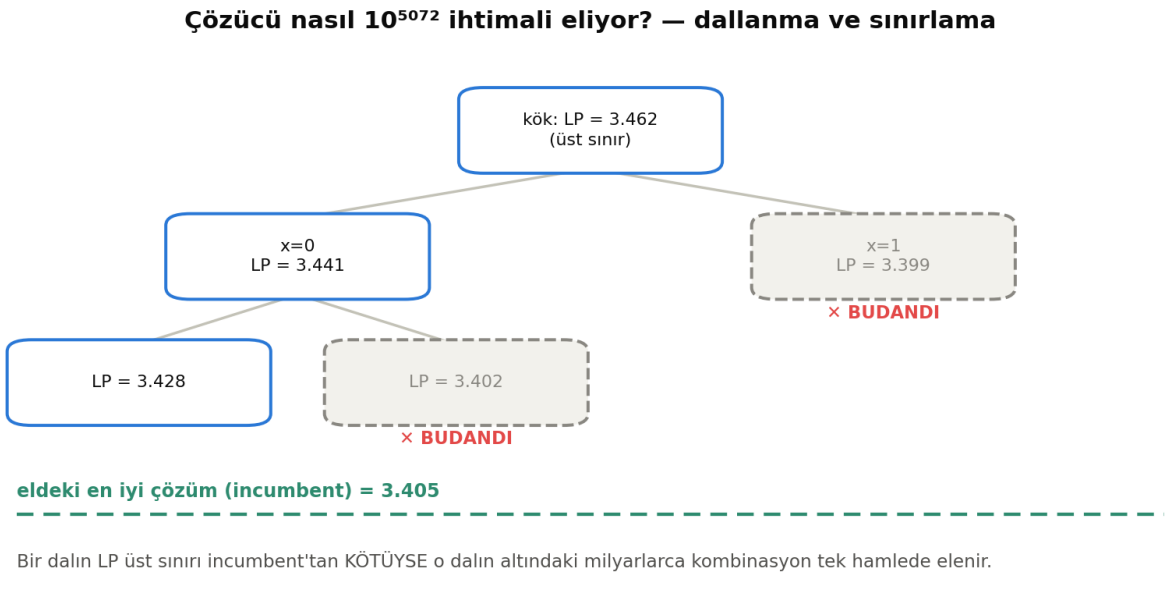
Çözücü (solver), matematiksel bir modeli alıp optimal çözümü bulan motordur. Kaba kuvvet kullanmaz — dört tekniği birlikte çalıştırır.

5.1 Doğrusal gevşeme (LP relaxation)

İlk adım şaşırtıcıdır: çözücü problemi **kolaylaştırır**. $x \in \{0,1\}$ şartını gevşetip $0 \le x \le 1$ yapar. Bu yeni problem artık bir doğrusal programdır ve simpleks/iç-nokta yöntemleriyle **çok hızlı** çözülür.

> **Neden işe yarıyor?**
> Gevşetilmiş problemin optimumu, gerçek problemin optimumundan **daha iyi ya da eşittir** — çünkü daha az kısıt vardır. Yani size bir **üst sınır** verir: "gerçek cevap bundan daha yüksek olamaz." Bu sınır, ilerideki budamaların temelidir.

5.2 Dallanma ve sınırlama (branch & bound)



Şekil 5.1 — Bir dalın üst sınırı eldeki en iyi çözümden kötüyse, o dalın altındaki tüm kombinasyonlar tek hamlede elenir.

Gevşemede $x = 0{,}63$ gibi kesirli bir değer çıkarsa çözücü ikiye dallanır: $x = 0$ ve $x = 1$. Her dalda gevşeme yeniden çözülür. Sihir burada:

1. Elde bir **uygun çözüm** (incumbent) varsa, değeri bilinir. Örneğin 3.405.000.
2. Bir dalın LP üst sınırı 3.399.000 çıkarsa, o dalın altında 3.405.000'i geçen bir çözüm **olamaz**.
3. O dal komple budanır — altındaki milyarlarca kombinasyon hiç incelenmeden elenir.
4. Böylece 10^{5072} ihtimalin ezici çoğunluğu **hiç ziyaret edilmeden** dışlanır.

5.3 Kesme düzlemleri (cutting planes)

Çözücü, kesirli çözümü dışarıda bırakan ama hiçbir geçerli tamsayı çözümü kesmeyen ek eşitsizlikler türetir. Gevşeme sıkılaşır, üst sınır düşer, budama hızlanır. Bunlar modelde yazmadığınız, çözücünün kendi ürettiği kısıtlardır.

5.4 Sezgiseller (heuristics)

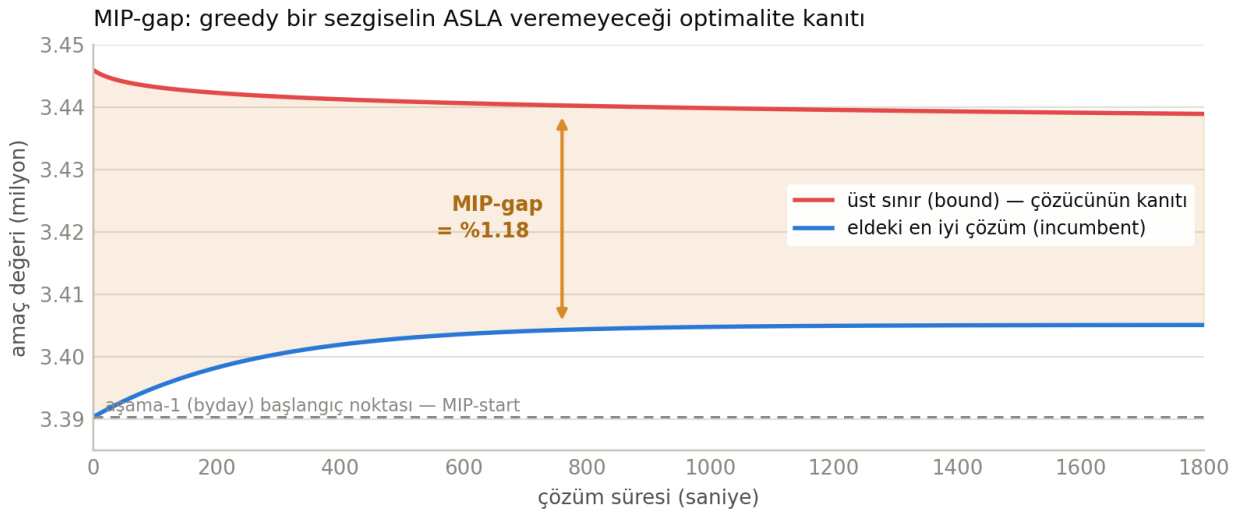
İyi bir incumbent'ı **erken** bulmak kritiktir — çünkü budama ancak elde bir karşılaştırma noktası varken çalışır. Çözücüler bunun için RINS, local branching, feasibility pump gibi yöntemler kullanır. Sizin kodunuz bunu bilinçli ayarlıyor:

`src/08_fullweek_polish.py` · satır 412–414 · sezgisel bütçesinin artırılması

```
412     # incumbent çevresi sezgisellere (RINS vb.) daha çok bütçe: warm-start'lı büyük
413     # MIP'te iyileştirmeler çoğunlukla sezgisellerden gelir (varsayılan 0.05)
414     CFG["highs_extra_options"] = {"mip_heuristic_effort": 0.3}
```

Varsayılan 0,05'ten 0,3'e çıkarılmış. Gerekçe kodda yazılı: warm-start'lı büyük MIP'te iyileştirmeler çoğunlukla incumbent çevresinde çalışan sezgisellerden gelir.

5.5 MIP-gap — çözücünün asıl armağanı



Şekil 5.2 — Alt (incumbent) ve üst (bound) sınır birbirine yaklaşır; aradaki fark gap'tir. Grafik şematiktir; nihai değerler gerçek koşudandır.

`src/03_model.py` · satır 939–945 · gap'in hesaplanması

```
939     gap = None
940     try:
941         bf, bp = res.best_feasible_objective, res.best_objective_bound
942         if bf not in (None, 0) and bp is not None:
943             gap = abs(bp - bf) / max(1e-9, abs(bf))
944     except Exception:
945         pass
```

Gap = |üst sınır − eldeki en iyi| / |eldeki en iyi|. Sizin teslim koşunuzda bu değer **%1,18**'dir ve şu anlama gelir:

> **"Bulduğum çözüm, teorik en iyinin en fazla %1,18 altında."**
>
> Bu cümle, elle yazılmış hiçbir açgözlü algoritmanın kuramayacağı cümledir. Greedy bir sezgisel size bir tarife verir — ama o tarifenin optimale yakın mı yoksa çok uzak mı olduğu konusunda **hiçbir bilgi** vermez. `08_fullweek_polish.py`'nin tüm varlık sebebi bu kanıtı üretebilmektir.

6 Hangi çözücü, neden?

6.1 Dört seçenek

Çözücü	Tür	Hız sınıfı	Lisans	Not
Gurobi	Ticari	En hızlı sınıf	Ücretli (akademik ücretsiz var)	Endüstri standardı; kurumsal lisans pahalı
CPLEX (IBM)	Ticari	Gurobi'ye yakın	Ücretli	Uzun geçmişli, olgun
HiGHS	Açık kaynak	Ticarilere en yakın açık seçenek	**Yok** — `pip install highspy`	Aktif geliştirilen modern çözücü
CBC	Açık kaynak	Belirgin biçimde yavaş	Yok	Eski nesil; büyük MIP'lerde zorlanır

6.2 Neden HiGHS seçildi?

1. **Lisans engeli yok.** Jüri `pip install -r requirements.txt` deyip kodu kendi makinesinde koşturabilir. Gurobi olsaydı lisans anahtarı gerekirdi ve sonuç bağımsız olarak doğrulanamazdı.
2. **Determinizm destekliyor.** Tek iş parçacığı ve sabit tohum ile aynı girdi her makinede aynı sonucu verir.
3. **Yeterince hızlı.** 100 binlik ikili değişkenli modeli warm-start ile 1.800 saniyede %1,18 gap'e getiriyor.
4. **Pyomo ile birinci sınıf desteği var** (`appsi_highs` arayüzü).

`src/03_model.py` · satır 917–931 · çözücünün kurulumu

```
917 def solve_model(m):
918     """appsi Highs ile çöz; zaman/gap sınırları + dürüst durum raporu."""
919     import time
920     from pyomo.contrib.appsi.solvers import Highs
921     opt = Highs()
922     opts = {"threads": 1, "random_seed": 0}    # determinizm (makineler arası tekrar-üretim)
923     opts.update(CFG.get("highs_extra_options") or {})      # ops. ek HiGHS ayarları (08 kullanır)
924     opt.highs_options = opts
925     if CFG["time_limit"] is not None:
926         opt.config.time_limit = CFG["time_limit"]
927     if CFG["mip_gap"] is not None:
928         opt.config.mip_gap = CFG["mip_gap"]
929     opt.config.load_solution = False
930     if CFG["warmstart"] and opt.warm_start_capable():
931         opt.config.warmstart = True
```

`threads: 1` performansı düşürmüyor mu?

Evet, düşürüyor. Ama çok çekirdekli MIP çözümü **deterministik değildir** — iş parçacıkları düğümleri farklı sırayla işler ve aynı girdiyle farklı çözümler dönebilir. Yarışma teslimi için "jüri koştuğunda aynı sayıyı görmeli" kısıtı, hız kaygısının önüne konmuştur.

6.3 Bu ayrımın somut faydası

Çözücüyü değiştirmek için model kodunda **tek satır** yeterlidir:

`src/03_model.py` · satır 97–99 · çözücü seçimi tek anahtarda

```
97     # --- çözücü ------------------------------------------------------------
98     "solver": "appsi_highs",
99     "time_limit": None,     # saniye (None = sınırsız); tam veride önerilir
```

Kısıtlar, amaç fonksiyonu, `build_model()`'in 400 satırı — hiçbiri değişmez. Model doğrudan HiGHS'in C++ API'sine yazılmış olsaydı, çözücü değiştirmek projeyi baştan yazmak demek olurdu.

7 `appsi` katmanı — ve 08 neden buna bağımlı

Kodunuzda düz `SolverFactory("highs")` yok; şu var:

`src/03_model.py` · satır 919–924 · appsi arayüzü

```
919     import time
920     from pyomo.contrib.appsi.solvers import Highs
921     opt = Highs()
922     opts = {"threads": 1, "random_seed": 0}    # determinizm (makineler arası tekrar-üretim)
923     opts.update(CFG.get("highs_extra_options") or {})      # ops. ek HiGHS ayarları (08 kullanır)
924     opt.highs_options = opts
```

appsi = *Auto-Persistent Pyomo Solver Interface*. Klasik arayüzden farkı: her çözümde diske bir `.lp`/`.mps` dosyası yazıp çözücüyü ayrı süreç olarak başlatmak yerine, **modeli çözücünün belleğinde canlı tutar** ve doğrudan C++ API'sini çağırır.

	Klasik (dosya tabanlı)	**appsi (kalıcı)**
Model aktarımı	Her çözümde `.lp` dosyası yaz/oku	Bellekte, doğrudan API
100 bin değişkenli modelde	Ciddi ek maliyet	Neredeyse sıfır
MIP-start (warm start)	Genellikle desteklenmez / kısıtlı	**`Highs.setSolution()`** olarak geçer
Ardışık çözümler	Her seferinde sıfırdan	Model korunur, sadece değişen kısım

08_fullweek_polish.py'nin tüm yöntemi buna bağlı

`CFG["warmstart"] = True` ayarı, appsi katmanında HiGHS'in `setSolution()` çağrısına dönüşür. Yani "aşama-1 çözümünü tam-hafta modeline başlangıç noktası olarak ver" fikri **ancak bu arayüzle mümkündür**. Dosya tabanlı bir arayüzde MIP-start'ı bu şekilde geçiremezdiniz — ve o zaman ne %+14.752'lik ek iyileşme ne de %1,18'lik global gap elde edilebilirdi.

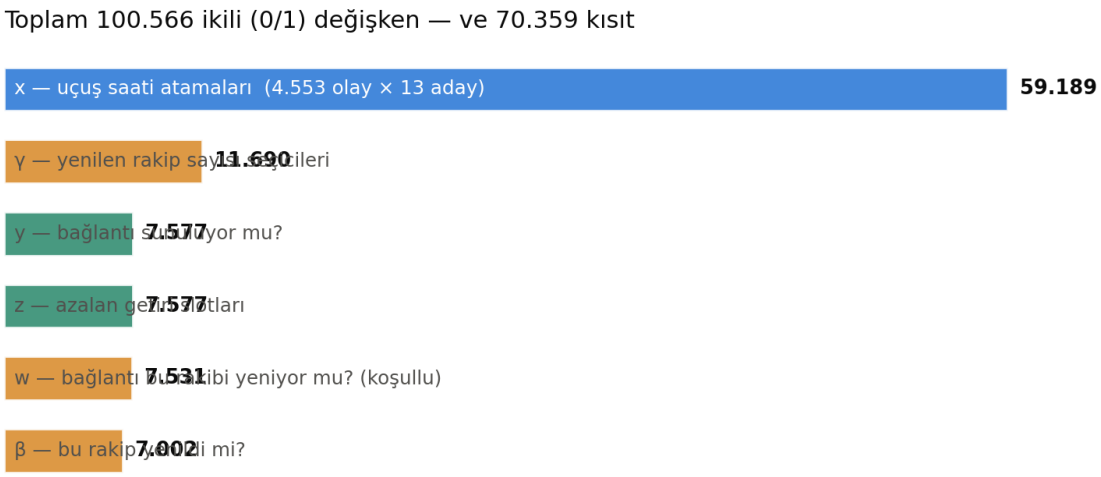
Nitekim `03_model.py`'deki şu yorum da buradan geliyor: *"appsi Highs MIP-start'ı kullanmıyor (yalnız LP bazı) — etkisiz."* 03 için etkisizdir çünkü 03 warm-start'ı kapalı koşar; 08'de `config.warmstart = True` ile gerçekten devreye girer.

8 Modelin gerçek boyutu

8.1 Örnek (instance) büyüklüğü

Ölçüt	Değer	Açıklama
Operasyon günü	**7**	Veriden okunur, hardcode değil
Pazar-gün grubu	**4.660**	Amaç fonksiyonunun toplandığı birim
Bağlantı (π)	**7.577**	Modele giren TK bağlantı adayı
Uçuş olayı	**5.376**	Karar birimi (uçuş no × rol × saat)
— ayarlanabilir	**4.553**	Her birinin 13 seçeneği var
— sabit	823	Rotasyon partneri yok → kımıldamaz
Rotasyon çifti	330	A2 kısıtının kurulduğu (gidiş, dönüş) ikilisi
Modellenmeyen kova kullanımı	79	ρ'suz pazarların F'deki sabit dolulukları

8.2 100.566 ikili değişken — dökümü



Şekil 8.1 — Değişkenlerin %59'u uçuş saati atamaları; gerisi ödül mantığını doğrusallaştıran yardımcı göstergeler.

Değişken	Sayı	Ne sorar?
`x[olay, saat]`	**59.189**	Bu uçuş olayı bu saate mi atandı? (4.553 × 13)
`γ[pazar, sayı]`	**11.690**	Bu pazarda tam olarak kaç rakip yenildi? (seçici)
`y[π]`	**7.577**	Bu bağlantı sunuluyor mu?
`z[pazar, slot]`	**7.577**	Azalan getiri slotu dolu mu?
`w[π, rakip]`	**7.531**	Bu bağlantı bu rakibi yeniyor mu? (koşullu olanlar)
`β[pazar, rakip]`	**7.002**	Bu rakip yenildi mi?
TOPLAM	**100.566**	—

Kısıt sayısı: **70.359**. Bunların kod tarafından ayrıca sayılan bazıları:

Kısıt grubu	Sayı	Not
G (düzenlilik)	4.503	4 grup pencerede ortak bant kuramadığı için atlandı
F (kova kapasitesi)	1.395	Yalnız bağlayıcı olabilen kovalar
A2 (rotasyon)	874 + 18	874 aynı-gün + 18 evren-ankoru; 623'ü brief-exact
A1 (tekil atama)	4.553	Her ayarlanabilir olay için tam bir tane
B1 (pencere)	15.154	Her bağlantı için alt + üst sınır

8.3 Ve sonuç

Ölçüt	Değer
Çözülen gün	7 / 7 (G gevşetilen: 0)
Kaydırılan uçuş hareketi	**1.067 / 4.554** (%23) — 3.487'si yerinde kaldı
Toplam kaydırma	20.640 uçuş-dk · ortalama **19 dk**
Sunulan bağlantı	6.966 / 7.577 aday
Yenilen rakip	4.914 / 8.832
Sıralaması yükselen pazar-gün	660 · değişmeyen 2.174 · gerileyen 24
BASELINE (mevcut tarife)	3.251.527
Aşama-1 (byday)	3.390.287
Aşama-2 (tam-hafta cilalama)	**3.405.039** (+14.752)
İYİLEŞME	**+153.512 (%+4,72)**
Global MIP-gap	**%1,18** — optimalite kanıtı
Bağımsız doğrulama	12 kontrolün tamamı PASS · A2 yeni ihlal: 0

Özetle

89.094 satırlık ham veriden, 10^{5072} büyüklüğünde bir karar uzayı doğdu. Pyomo bu uzayı 100.566 ikili değişken ve 70.359 kısıtla **tarif etti**; HiGHS dallanma-sınırlama ile **çözdü** ve çözümün optimale en fazla %1,18 uzak olduğunu **kanıtladı**. Sonuç: 1.067 uçuş hareketinin ortalama 19 dakika kaydırılmasıyla gelir-ağırlıklı rekabetçilikte %4,72 artış.

A Ek A — Tek sayfalık sayı künyesi

Kategori	Ölçüt	Değer
Girdi	Ham Excel satırı (4 dosya)	89.094
	Bağlantı tablosu satırı	57.317
	— THY (TK)	7.850
	— rakip	49.467
	Operasyon günü	7
	Yönlü O–D pazarı	1.408
	TK uçuş numarası	952
	Farklı taşıyıcı	88
Türetilmiş	`derived/` toplam satır	118.177
	TK bağlantı adayı	7.693
	ρ'su olmayan (dışlanan) pazar	48
	Rotasyon partneri olmayan uçuş no	176
Karar	Uçuş olayı (toplam)	5.376
	— ayarlanabilir	4.553
	— sabit	823
	Olay başına seçenek	13 (±60 dk / 10 dk)
	Tarife karar uzayı	$13^{4553} \approx 10^{5.072}$
Model	İkili değişken	100.566
	Kısıt	70.359
	Ham ikili uzay	$2^{100566} \approx 10^{30.273}$
Çözüm	Kaydırılan uçuş hareketi	1.067 (%23)
	Ortalama kaydırma	19 dk
	Sunulan bağlantı	6.966
	Yenilen rakip	4.914 / 8.832
	Baseline → Sonuç	3.251.527 → 3.405.039
	İyileşme	+153.512 (%+4,72)
	Global MIP-gap	%1,18
Altyapı	Modelleme	Pyomo 6.7+
	Çözücü	HiGHS (`appsi_highs`, `highspy`)
	Determinizm	threads=1, random_seed=0
	Süre sınırı	aşama-1: 150 s/gün · aşama-2: 1.800 s

B Ek B — İki konunun adım adım anlatımı

Bu ek, dokümandaki iki sayının *nereden geldiğini* sıfırdan, hiçbir ön bilgi varsaymadan anlatır.

B.1 "823 uçuş olayı neden kımıldatılamıyor?"

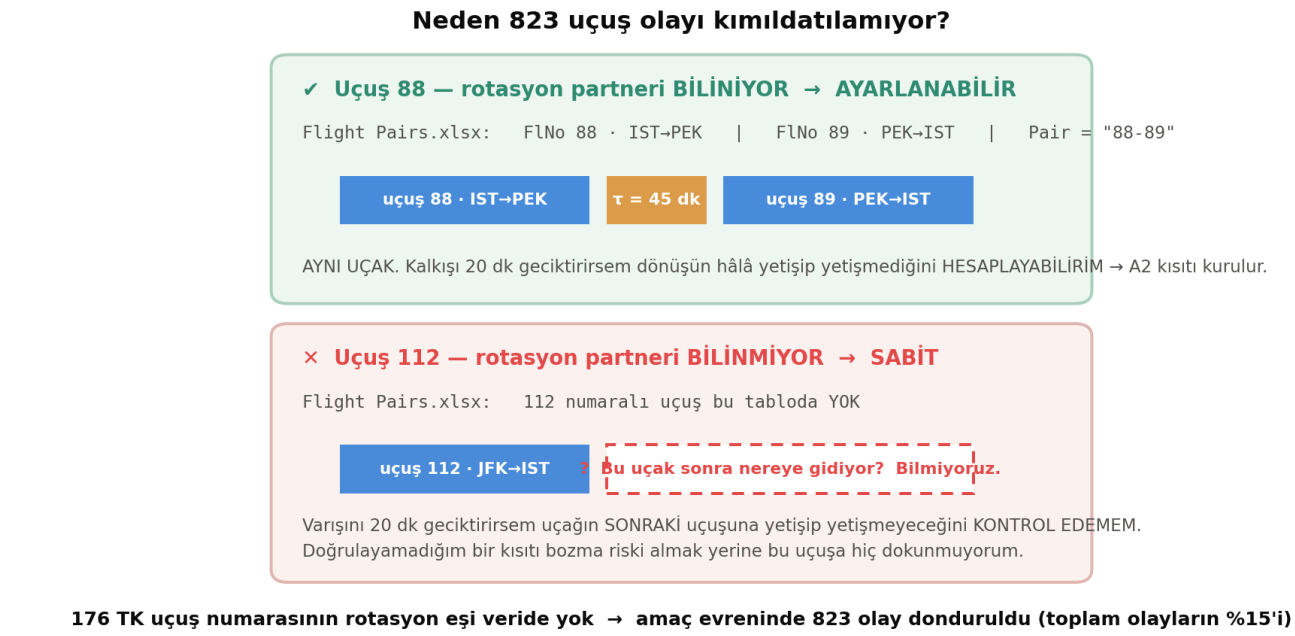
Önce: "uçuş olayı" ne demek?

Bir uçuş, IST'te ya **iner** ya **kalkar**. Bizim için karar birimi "uçuş" değil, **uçuşun IST'teki tek bir hareketidir**. Buna *olay* diyoruz. Bir olayın kimliği üç şeyden oluşur:

- hangi uçuş numarası (örn. 88)
- hangi rol (IST'e **varış** mı, IST'ten **kalkış** mı)
- hangi gün (1–7)

Modelde toplam **5.376 olay** var. Sorumuz şu: bunlardan 823'ünün saati neden değiştirilemiyor?

Cevap: uçak fiziksel bir nesnedir ve bir yere gidip geri dönmek zorundadır



Şekil B.1 — Rotasyon partneri bilinen bir uçuş oynatılabilir; bilinmeyen oynatılamaz.

Bir uçak IST'ten kalkar, bir yere gider, orada bir süre bekler ve **aynı uçak** geri döner. Bu turun süresi en az şu kadar olmalıdır:

Dönüş varışı − Gidiş kalkışı ≥ T_OB + τ + T_IB

T_OB — gidiş bacağının uçuş süresi
τ (tau) — hedef havalimanında yerde geçen minimum süre (45 dk: yolcu indirme/bindirme, yakıt, temizlik)
T_IB — dönüş bacağının uçuş süresi

Bu, modeldeki **A2 kısıtıdır**. Bunu sağlamayan bir tarife kâğıt üstünde vardır ama gerçekte uçurulamaz — uçak orada olmaz.

Somut örnek — kımıldayabilen uçuş

`Flight Pairs.xlsx` dosyasında şu satırlar var:

FlNo	Orig	Dest	Pair
88	IST	PEK	88-89
89	PEK	IST	88-89

Aynı `Pair` etiketi şunu söylüyor: **88 numaralı uçuşu yapan uçak, 89 numaralı uçuşla geri dönüyor.** Dolayısıyla 88'in kalkışını 20 dakika geciktirmek istediğimde, 89'un varışının hâlâ yetişip yetişmediğini **hesaplayabilirim**. Model bu kısıtı kurar, çözücü ona uyar. Bu uçuş **ayarlanabilir**.

Somut örnek — kımıldayamayan uçuş

Bağlantı tablosunda **112** numaralı bir TK uçuşu var (JFK → IST, gün 1'de 23:25'te iniyor). Ama `Flight Pairs.xlsx` dosyasında **112 numaralı uçuş hiç geçmiyor**. Yani elimizde şu bilgi yok:

- Bu uçuşu hangi uçak yapıyor?
- O uçak IST'e indikten sonra hangi uçuşa gidiyor?
- Varışını 20 dakika geciktirirsem, o uçak bir sonraki uçuşuna yetişebilir mi?

> **Bilmediğimiz bir kısıtı bozma riski almıyoruz**
>
> 112'yi yine de kaydırabilirdik — puan biraz artardı. Ama sonuçta **fiziksel olarak uygulanamayan** bir tarife üretmiş olabilirdik ve bunu fark etmemizin hiçbir yolu olmazdı, çünkü kontrol edecek veri yok.
>
> Proje bunun yerine ihtiyatlı yolu seçiyor: *doğrulayamadığın şeye dokunma.* Bu uçuşlar mevcut saatlerinde bırakılıyor ve model onları sadece "orada duran sabit uçuşlar" olarak görüyor — IST kapasitesinde yer kaplıyorlar ama karar değişkeni olmuyorlar.

Kaç tane, ve nereden 823 çıkıyor?

Adım	Sayı	Açıklama
Bağlantı tablosundaki TK uçuş numarası	**952**	Toplam farklı uçuş no
Flight Pairs'te karşılığı olan	776	Rotasyon eşi biliniyor → ayarlanabilir
Flight Pairs'te **olmayan**	**176**	Rotasyon eşi yok → sabit
Bunların amaç evreninde görüneni	172	4'ü yalnız ρ'suz pazarlarda uçuyor
Uçuş numarası başına ortalama olay	4,78	7 günün bir kısmında, tek rolde
Toplam sabit olay	**823**	172 × 4,78 ≈ 823
Toplam olay	5.376	Sabitlerin payı: **%15,3**

Bunun kodda karşılığı

`src/02_data_prep.py` · satır 288–295 · listenin üretilmesi

```
288     # === Rotasyon (Flight Pairs) — eksik partner = ayarlanamaz işareti =========
289     pair = pair.copy()
290     tk_flnos = set(tk["FlNo1"]).union(set(tk["FlNo2"]))
291     pair_flnos = set(pair["FlNo"])
```

```
292     no_partner = sorted(tk_flnos - pair_flnos)
293     rot = pair[pair["FlNo"].isin(tk_flnos)].copy()
294     log(f"\n[Rotasyon] TK uçuş no: {len(tk_flnos)} | Flight Pairs'te eşi olan: {len(tk_flnos & pair_flnos)}")
295     log(f"  Partnersiz TK uçuş no: {len(no_partner)} -> politika: 'ayarlanamaz/sabit' kabul (kısıt eklenmez)")
```

`02_data_prep.py` bu 176 uçuş numarasını bulup `derived/tk_flights_without_partner.csv` dosyasına yazar.

`src/03_model.py` · satır 236–247 · listenin uygulanması

```
236     def get_event(flno, role, cur_abs, day):
237         cur = snap(cur_abs)
238         key = (int(flno), role, cur)
239         if key not in events:
240             adj = int(flno) not in fixed_flnos          # rotasyon partneri yoksa SABIT (docs D6)
241             if adj:
242                 cands = [cur + k * delta for k in range(-(wd // delta), wd // delta + 1)]
243             else:
244                 cands = [cur]
245             events[key] = dict(id=len(events), flno=int(flno), role=role,
246                                cur=cur, day=int(day), adjustable=adj, cands=cands)
247         return events[key]["id"]
```

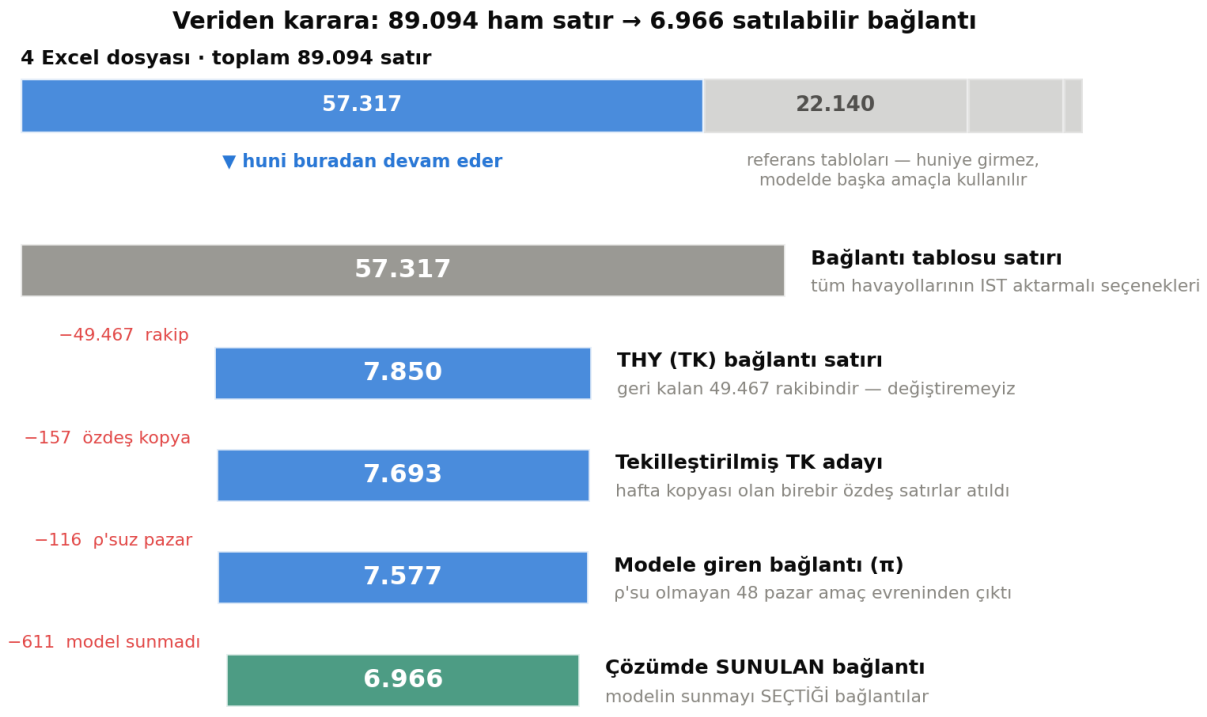
`03_model.py` bu listeyi okur. **Satır 240** kararı verir: uçuş numarası listede varsa `adj = False`. **Satır 244** sonucu uygular: ayarlanamaz olayın aday listesi `cands = [cur]` olur — yani tek seçenek, mevcut saatin kendisi. Çözücü başka bir değer seçemez çünkü seçecek başka değer yoktur.

Bunun bedeli ne?

Dürüst cevap: **puan kaybı**. Bu 823 olay serbest bırakılsaydı model daha iyi bir sonuç bulurdu. Arama uzayı da $13^{823} = 10^{917}$ kat büyürdü. Yani burada bilinçli bir takas var: *daha yüksek puan* yerine *uygulanabilirliği kanıtlanmış tarife*. Organizatör eksik rotasyon verisini sağlarsa bu 823 olay tek satır değişiklikle serbest bırakılabilir.

B.2 Huni figürü (Şekil 2.1) tam olarak neyi gösteriyor?

Bu figür **tek bir hikâyeyi** anlatır: elimize gelen ham veri, adım adım süzülerek sonunda "yolcuya satılabilir bağlantı" listesine dönüşüyor. Her çubuk bir öncekinin **alt kümesidir** — yani her satır, üstündeki satırın içinden çıkar.



Şekil B.2 — Huninin düzeltilmiş hâli: üstteki şerit girdi dosyalarının dağılımını, alttaki çubuklar ise gerçek süzülmeyi gösteriyor.

Üstteki şerit: 89.094 satır neyin toplamı?

Burada **dikkat edilmesi gereken bir ayrım** var. 89.094, dört Excel dosyasının satır toplamıdır ve bunların hepsi "bağlantı" değildir:

Dosya	Satır	Bu satırlar ne?	Huniye girer mi?
`O&D Rakip Bağlantı Tablosu`	**57.317**	Her satır bir aktarmalı yolculuk seçeneği	**EVET** — huninin girdisi
`change_ranking_input.xlsx`	22.140	Ödül tablosu: (N rakip, b sıra, s sıra) → puan	Hayır — **başvuru tablosu**
`Yolcu Verisi_masked.xlsx`	8.075	Pazar → gelir ağırlığı ρ	Hayır — **çarpan tablosu**
`Flight Pairs.xlsx`	1.562	Uçuş no → rotasyon eşi	Hayır — **kısıt verisi**

Yani 89.094 → 57.317 bir "eleme" DEĞİLDİR

Diğer 31.777 satır atılmıyor — modelde sonuna kadar kullanılıyorlar; sadece *bağlantı adayı* değiller. Ödül tablosu puan hesabında, ρ tablosu amaç fonksiyonunda, Flight Pairs ise A2 rotasyon kısıtında iş görür. Figürün ilk şeridi bu yüzden **eleme oku değil, dağılım şeridi** olarak çizilmiştir.

Çubuk 1 — 57.317: bağlantı tablosunun tamamı

Bu tablonun her satırı şu bilgiyi taşır: "*Şu havayolu, şu günde, şu gelen uçuşla IST'e geliyor; şu giden uçuşla şu şehre devam ediyor.*" Yani bir satır = **bir yolculuk seçeneği**. 57.317 satır, 88 farklı havayolunun 1.408 pazardaki 7 günlük tüm aktarma seçenekleridir.

Çubuk 2 — 7.850: bunun THY'ye ait olanı

`Cr1` sütunu "TK" olan satırlar. Kalan **49.467 satır rakiplerindir.** Onları **değiştiremeyiz** — rakip havayolunun tarifesine müdahale edemeyiz. Ama onlar kaybolmaz: her rakibin o pazardaki **en iyi yolculuk süresi** (T_comp) hesaplanır ve bizim kıyas ölçümüz olur. "Rakip yendik mi?" sorusu tam olarak bununla cevaplanır.

Çubuk 3 — 7.693: 157 özdeş kopya atıldı

Veride şöyle bir tuhaflık var: **aynı operasyon günü** ("Gün 1") farklı pazarlarda farklı takvim tarihlerine yazılmış. Bir satırda 4 Ağustos Pazartesi, başka satırda 11 Ağustos Pazartesi. İkisi de "Gün 1"dir ve aynı fiziksel uçuşu anlatır — sadece bir hafta kaymış görünür.

`02_data_prep.py` bunları hizalar (kaymış olanları 7 gün geri çeker). Hizalama sonrası **157 satır**, başka bir satırın **birebir aynısı** hâline gelir: aynı pazar, aynı gün, aynı iki uçuş numarası, aynı iki saat.

Neden atmak zorundayız?

Aynı ürünü iki kez saymak, aktarma ödülünü **iki kez** yazdırır. Bu hem mevcut tarifenin (baseline) puanını hem de optimize sonucun puanını şişirir — yani elde ettiğimiz "iyileşme" sahte olur. Dikkat: **yalnız tüm alanları özdeş** olanlar atılır; saati gerçekten farklı olan satırlar korunur, çünkü onlar gerçekten farklı uçuşlardır.

Çubuk 4 — 7.577: 116 bağlantı amaç evreninden çıktı

Amaç fonksiyonu şöyle: **ρ × (aktarma ödülü + sıralama ödülü)**. Buradaki ρ, pazarın gelir ağırlığıdır ve `Yolcu Verisi` dosyasından gelir. Ama bağlantı tablosundaki **48 pazarın** bu dosyada karşılığı yok — ρ'ları bilinmiyor.

Üç seçenek vardı:

Seçenek	Ne olurdu?	Karar
ρ = 0 varsay	O pazarları "değersiz" ilan etmek olurdu	Reddedildi
ρ = ortalama varsay	**Veri uydurmak** olurdu	Reddedildi
Amaç evreninden çıkar	Puanlamaya girmezler; ama uçuşları IST kapasitesinde sayılmaya devam eder	**Seçildi**

Kritik nokta: puanlamadan çıktılar, FİZİKTEN çıkmadılar

Bu 48 pazarın uçakları hâlâ havada ve hâlâ IST pistini kullanıyor. Onları kapasite hesabından da çıkarsaydık, gerçekte olmayan bir boşluk yaratır ve model oraya uçuş yerleştirirdi. Kod bunu `extra_use` mekanizmasıyla önler: bu uçuşlar **karar değişkeni olmadan, sabit doluluk olarak** kova sayacına eklenir (79 kova etkileniyor).

Çubuk 5 — 6.966: modelin sunmayı SEÇTİĞİ bağlantılar

Buraya kadar olan tüm daralmalar **veriyle** ilgiliydi. Bu son adım farklıdır: burası **modelin kararıdır** (bu yüzden çubuk yeşil). 7.577 adaydan 611'i sunulmadı. İki nedeni var:

1. **Pencereye sokulamıyor.** Bağlantının boşluğu 60–300 dakika aralığına ancak ±60 dakikalık kaydırmalarla girebilir. Bazı bağlantılarda gelen ve giden uçuş arasındaki mesafe o kadar büyük ki, iki uçuşu da sonuna kadar kaydırsanız bile pencereye giremiyor.
2. **E1 yön dengesi için bilinçli olarak sunulmuyor.** Model, bir pazarın gidiş ve dönüş yönlerinin bağlantı sayılarını dengeli tutmak zorunda ($|n_{od} - n_{do}| \le 0{,}20 \times$ toplam). Bir yönde 20, ters yönde 8 bağlantı varsa, 20'nin hepsini sunmak dengeyi bozar — model bilerek bir kısmını sunmaz.

İkinci neden ilk bakışta garip gelir: "neden kârlı bir şeyi sunmuyorsun?"

Çünkü tek yönü şişirip dönüşü ihmal eden bir tarife kâğıt üstünde puan kazanır ama **gerçekte satılabilir bir ürün değildir**. Yolcu gidiş-dönüş ister. E1 kısıtı bu gerçekliği modele sokar. (Bu ayrıntı `08_fullweek_polish.py`'de de kritiktir: warm-start'ta hangi bağlantıların sunulduğu **uygunluktan türetilemez**, byday'in gerçek kararından okunmalıdır.)

Huninin özeti — tek cümlede her satır

Satır	Sayı	Tek cümlede
Ham girdi	89.094	Dört Excel dosyasının toplam satırı (hepsi bağlantı değil)
Bağlantı tablosu	57.317	Tüm havayollarının IST aktarma seçenekleri
THY bağlantısı	7.850	Bunun bize ait olanı — geri kalanı rakip
Tekilleştirilmiş	7.693	Aynı ürünü iki kez saymamak için 157 kopya atıldı
Modele giren	7.577	Gelir ağırlığı bilinen pazarların bağlantıları
Sunulan	**6.966**	Modelin, tüm kısıtları sağlayarak sunmayı seçtiği bağlantılar

Ve nihayetinde optimizasyonun ölçtüğü şey bu son satırdır: bu 6.966 bağlantının toplam yolculuk süreleri, rakiplerinkinden ne kadar iyi ve bu pazarlar THY için ne kadar değerli.